

คู่มือโครงการ การเรียบเรียงข้อความภาษาไทย

รายละเอียดระบบ ไฟล์ข้อมูล อัลกอริทึมการเตรียมชุดข้อมูล การ fine-tune การประเมินผล และ CLI สำหรับทดลองเรียบเรียงข้อความ

ข้อสรุปที่ต้องทราบก่อนอ่าน: ชุดข้อมูลที่ใช้ฝึก mT5 เป็นชุดสรุปข้อความ ไม่ใช่ชุดคู่ paraphrase โดยตรง ส่วน CLI ปัจจุบันใช้ Typhoon2 3B ซึ่งเป็นโมเดล instruction ที่ดาวน์โหลดแยกและยังไม่ได้ fine-tune ด้วยข้อมูลในโครงการ

สารบัญและภาพรวม

- 1. บทสรุปและขอบเขตของระบบ
- 2. สถาปัตยกรรมและเส้นทางข้อมูล
- 3. รายละเอียดไฟล์ทุกส่วน
- 4. ชุดข้อมูลและการเตรียมข้อมูล
- 5. อัลกอริทึมการฝึก mt5
- 6. การประเมินผลและหลักฐาน
- 7. CLI, Typhoon2 และ output guard
- 8. วิธีติดตั้งและรัน
- 9. ข้อจำกัด ความเสี่ยง และงานต่อ
- 10. ที่มาและการอ้างอิง

คำศัพท์: source หมายถึงข้อความต้นฉบับ, target/reference หมายถึงข้อความเป้าหมายที่อยู่ในข้อมูลฝึกหรือใช้เทียบ metric, inference คือการให้โมเดลสร้างคำตอบจากข้อความใหม่, weak supervision คือการใช้ label/target ที่ไม่ตรงกับงานเป้าหมายอย่างสมบูรณ์

1. บทสรุปและขอบเขต

repository นี้เริ่มจากการทดลอง fine-tune google/mt5-small ด้วยตัวอย่างประเภท summarization จาก airesearch/wangchanx-seed-free-synthetic-instruct-thai-120k โดยจับ context เป็น source และ output เป็น target แล้วคัดตัวอย่างด้วยคะแนนและอัตราส่วนความยาว

เมื่อลองให้ mt5 เรียบเรียงประโยคใช้งานจริง พบการวนซ้ำ ปลอ่ย special token และเปลี่ยนข้อเท็จจริง เช่น “เที่ยง” กลายเป็น “เที่ยงคืน” จึงไม่ใช่ mt5 เป็นโมเดล CLI ปัจจุบัน แต่เก็บ model/report ไว้เป็น baseline และเลือก Typhoon2 Llama 3.2 3B Instruct สำหรับ CLI หลังทดสอบ smoke test กับสองตัวอย่าง

ส่วนระบบ	โมเดล	วัตถุประสงค์	สถานะ
Training / evaluation	google/mt5-small	ทดลองเรียนรู้ source→target จาก weakly supervised summarization pairs	มี weights, report และคะแนนบน test แต่ไม่ตรง paraphrase โดยตรง
CLI inference	typhoon-ai/llama3.2-typhoon2-3b-instruct	รับ instruction ภาษาไทยและสร้างข้อความเรียบเรียง	โหลดจาก Hugging Face ครั้งแรก; ไม่ได้ fine-tune ด้วย data ใน repository

อย่าสรุปเกินหลักฐาน: ตัวอย่าง CLI สองประโยคเป็น smoke test เท่านั้น ไม่ใช่ benchmark และไม่พิสูจน์ว่าโมเดลรักษาความหมายได้ทุกกรณี

2. สถาปัตยกรรมและเส้นทางข้อมูล

2.1 เส้นทางเตรียมข้อมูลและฝึก

Hugging Face 118,898 rows	prepare_data.py filter + dedupe + split	data/*.jsonl 3,893 / 486 / 486	train.py mT5 fine-tune	artifacts/ weights + evidence
-------------------------------------	--	---	----------------------------------	--

1. โหลด training split ของ dataset ต้นทาง
2. คัดเฉพาะ summarization, คำนวณอย่างน้อย 7, source/target ต้องไม่ว่าง, target/source ยาว 0.65–1.25 และ source ต้องไม่ซ้ำ
3. สุ่มแบบ deterministic ด้วย seed 42 แบ่ง train/validation/test เป็น 80/10/10 หลัง deduplicate
4. ฝึก mT5 ด้วย train และคำนวณ evaluation loss บน validation ทุก epoch
5. บันทึก model/checkpoint และ training_report.json

2.2 เส้นทางประเมิน baseline

evaluate.py โหลด artifacts/model ซึ่งเป็น mT5 ที่ fine-tune แล้ว สร้าง prediction สำหรับ data/test.jsonl ทั้ง 486 แถว จากนั้นคำนวณ chrF และ ROUGE-L และบันทึก report/predictions

2.3 เส้นทางใช้งานข้อความใหม่

PowerShell ข้อความ + mode	app.py chat template	Typhoon2 3B greedy generation	validate_output heuristic guard	พิมพ์ผลหรือ fallback
-------------------------------------	--------------------------------	--	---	--------------------------------

CLI ไม่ได้อ่าน weights ใน artifacts/model แต่โหลด typhoon-ai/llama3.2-typhoon2-3b-instruct แยกผ่าน Hugging Face cache เพราะการทดลอง mT5 เติบโตผ่านเกณฑ์การเรียบเรียงตัวอย่าง

3. รายละเอียดไฟล์และโฟลเดอร์

ไฟล์/โฟลเดอร์	หน้าที่และข้อมูลสำคัญ
app.py	จุดเริ่มต้น CLI สำหรับผู้ใช้; รับ --mode, --text, --max-new-tokens; โหลด Typhoon2; สร้าง chat prompt; generate; ตรวจสอบ output; พิมพ์คำตอบหรือคืน source
prepare_data.py	โหลด dataset, กรอง/ตัดซ้ำ, สุ่มแบ่งข้อมูล, เขียน JSONL และ manifest พร้อม SHA-256
train.py	fine-tune mT5 แบบ sequence-to-sequence บน train/validation splits และเขียน model/report
evaluate.py	สร้าง mT5 predictions บน test split คำนวณ chrF/ROUGE-L และบันทึกผลตรวจสอบรายตัวอย่าง
README.md	คำแนะนำเริ่มต้น ข้อจำกัดและคำสังรั้น (คู่มือนี้ขยายรายละเอียดเพิ่มเติม)
requirements.txt	dependencies หลัก เช่น datasets, torch, transformers, sentencepiece
requirements-train.txt	อ้าง requirements.txt และเพิ่ม accelerate, rouge-score, sacrebleu สำหรับ train/evaluate
data/manifest.json	หลักฐาน provenance, dataset fingerprint, เงื่อนไขคัด, จำนวน rejected/accepted, split seed, row counts, SHA-256 และข้อจำกัด
data/train.jsonl	3,893 ตัวอย่างสำหรับอัปเดตน้ำหนัก mT5
data/validation.jsonl	486 ตัวอย่างสำหรับคำนวณ eval loss/เลือก checkpoint ระหว่าง train; ไม่ใช่ test
data/test.jsonl	486 ตัวอย่าง holdout สำหรับประเมินหลังฝึก; ห้ามนำไปฝึก
artifacts/model/	mT5 fine-tuned model สำหรับ evaluate.py; มี config, generation config, weights, tokenizer และ training args
artifacts/checkpoints/	checkpoint/trainer state จาก Hugging Face Trainer; checkpoint มี model, scheduler, RNG, tokenizer และ state บางไฟล์
artifacts/checkpoints/checkpoint-487/optimizer.pt	optimizer state ขนาดประมาณ 2.29 GB; ถูก ignore และเก็บ local-only เพราะเกิน GitHub LFS Free/Pro ต่อไฟล์ 2 GB
artifacts/training_report.json	พารามิเตอร์และ metrics ของการฝึก mT5 พร้อม hashes ของ train/validation
artifacts/evaluation_report.json	metrics บน test split 486 แถวและคำอธิบายความหมาย/ข้อจำกัดของ metric
artifacts/predictions.jsonl	source, reference, model output และ source index สำหรับตรวจสอบ output รายแถว
.gitignore	ไม่ track venv, cache, secret .env, Python cache และ optimizer.pt
.gitattributes	กำหนด *.safetensors ให้ใช้ Git LFS

.env	ค่าตั้งค่าส่วนตัว/secret ถ้ามี; ถูก ignore ห้ามเผยแพร่ค่าในไฟล์
.venv/	Python virtual environment ในเครื่อง ไม่ใช่ source และไม่ควรร commit
.git/	ประวัติ/version control ของ repository ไม่ใช่ข้อมูล runtime
__pycache__/	ไฟล์ bytecode ที่ Python สร้างอัตโนมัติ; ไม่ใช่ source

3.1 ไฟล์ย่อยของ model และ checkpoint

ไฟล์	บทบาท
artifacts/model/config.json	โครงสร้าง mT5 เช่น จำนวน layer, ขนาด hidden state, vocabulary และ special token IDs
artifacts/model/generation_config.json	ค่าเริ่มต้นสำหรับการ generate ของ model ที่บันทึกไว้
artifacts/model/model.safetensors	น้ำหนัก mT5 ที่ fine-tune แล้ว; ใช้โดย evaluate.py ไม่ใช่ Typhoon CLI
artifacts/model/special_tokens_map.json	mapping ของ pad/eos/unk และ special tokens
artifacts/model/spiece.model	SentencePiece vocabulary/model ที่ tokenizer ของ mT5 ใช้แบ่งข้อความ
artifacts/model/tokenizer.json	serialised tokenizer backend สำหรับการ tokenize/decode
artifacts/model/tokenizer_config.json	ชนิด tokenizer และค่าตั้งที่ใช้โหลด tokenizer กลับ
artifacts/model/training_args.bin	Hugging Face TrainingArguments ที่ serialize จากรอบฝึก
artifacts/checkpoints/trainer_state.json	trainer state ที่ระดับ output directory ใช้บันทึก step/log/checkpoint metadata
artifacts/checkpoints/checkpoint-487/config.json	configuration ของ model ใน checkpoint-487
artifacts/checkpoints/checkpoint-487/generation_config.json	ค่า generation ที่บันทึกพร้อม checkpoint
artifacts/checkpoints/checkpoint-487/model.safetensors	weights ณ checkpoint นี้; ไฟล์เนื้อหาซ้ำกับ model.safetensors ใน artifacts/model
artifacts/checkpoints/checkpoint-487/optimizer.pt	สถานะ optimizer สำหรับ resume training; ไม่จำเป็นต่อ inference และถูก ignore เพราะ 2.29 GB
artifacts/checkpoints/checkpoint-487/rng_state.pth	สถานะ random number generators เพื่อ resume ให้ต่อเนื่อง
artifacts/checkpoints/checkpoint-487/scheduler.pt	สถานะ learning-rate scheduler สำหรับ resume training
artifacts/checkpoints/checkpoint-487/special_tokens_map.json	mapping special tokens ของ tokenizer ใน checkpoint
artifacts/checkpoints/checkpoint-487/spiece.model	SentencePiece model ที่ checkpoint ใช้
artifacts/checkpoints/checkpoint-487/tokenizer.json	serialised tokenizer backend ณ checkpoint

artifacts/checkpoints/checkpoint-487/tokenizer_config.json	ค่าตั้งสำหรับสร้าง tokenizer จาก checkpoint
artifacts/checkpoints/checkpoint-487/trainer_state.json	global step, log history และข้อมูลควบคุมการฝึกของ checkpoint
artifacts/checkpoints/checkpoint-487/training_args.bin	TrainingArguments ที่ serialize ณ checkpoint

4. ชุดข้อมูลและอัลกอริทึมการเตรียม

4.1 ที่มาและ schema

แหล่งข้อมูลคือ airesearch/wangchanx-seed-free-synthetic-instruct-thai-120k ซึ่ง manifest ระบุ 118,898 rows ใน train split, fingerprint 24a35b6a961cfde3, license MIT ตาม dataset card และ citation งาน ACL SRW 2024

ตัวอย่างใน JSONL มี keys: source_text, target_text, instruction, rating, source_index. ใน source dataset ใช้ context เป็นข้อความต้นทางและ output เป็นคำตอบเป้าหมาย

ชนิดข้อมูลใน source	จำนวนแถว
conversation	24,865
multiple_choice	23,975
brainstorming	23,844
question_answering	23,366
summarization	22,848

4.2 ตัวกรอง

- ถ้า type != summarization ให้ reject
- ตัด whitespace ที่หัว/ท้าย source และ target; ค่าว่างอย่างใดอย่างหนึ่งให้ reject
- แปลง rating เป็น float และต้อง rating >= 7.0
- คำนวณ $r = \text{len}(\text{target}) / \text{len}(\text{source})$ ด้วยจำนวนตัวอักษร Unicode ของ Python; รับเฉพาะ $0.65 \leq r \leq 1.25$
- สร้าง SHA-256 จาก source ที่ trim แล้ว; source hash ที่เคยพบแล้วจะ reject
- เก็บ source/target/instruction/rating/index ไว้

```
for each source row:
    if type != summarization: reject
    source = trim(context); target = trim(output)
    if source or target is empty: reject
    if rating < 7: reject
    ratio = character_count(target) / character_count(source)
    if ratio outside [0.65, 1.25]: reject
    if sha256(source) already seen: reject
    accept(source, target, instruction, rating, source_index)
```

4.3 Split และหลักฐาน

คัดแล้ว 4,865 คู่; rejected ตามเหตุผล: ไม่ใช่ summarization 96,050, length ratio 12,550, คะแนนต่ำ 4,230, source ซ้ำ 1,203 รวม 114,033 แถว. จากนั้นใช้สำเนารายการที่ยอมรับ, random.Random(42).shuffle, ปิดจำนวน test/validation ด้วย $\text{round}(n \times 0.10)$, และใช้ส่วนที่เหลือเป็น train

Split	จำนวน	SHA-256 (ขั้นต้น)	ใช้ทำอะไร
train	3,893	569edc41e3d4...	คำนวณ gradient/update น้ำหนัก
validation	486	410d19713fa8...	คำนวณ eval loss ระหว่างฝึก
test	486	287f200bedd2...	ประเมินหลังฝึกเท่านั้น

การตีความ: length ratio ช่วยตัดคำตอบที่สั้นกว่ามาก แต่ไม่ได้รับรองว่าเป็น paraphrase; target อาจสรุป ตกหล่น หรือเพิ่มข้อเท็จจริง และไม่มี style labels สำหรับ formal/semiformal/casual

5. อัลกอริทึมการฝึก mT5

5.1 การแปลง input/target

สำหรับแต่ละแถว train/validation สคริปต์ประกอบ input เป็น instruction: {instruction}\ncontext: {source_text} แล้ว tokenize โดยตัดที่ 512 tokens. Target tokenize แยกด้วย text_target และตัดที่ 384 tokens. Data collator ทำ padding ให้ batch และตั้ง labels สำหรับ seq2seq loss

5.2 การเรียนรู้

mT5 เป็น encoder-decoder seq2seq. Encoder อ่าน token ของ instruction + source; decoder เรียนรู้ทำนาย target token ทีละตำแหน่งโดยใช้ teacher forcing. วัตถุประสงค์เป็น token cross-entropy: ลดความคลาดเคลื่อนระหว่าง token เป้าหมายและการกระจายความน่าจะเป็นที่โมเดลทำนาย

```
input_ids = tokenize("instruction: ...\ncontext: " + source)
labels    = tokenize(target)
for batch in train:
    logits = mT5(input_ids, labels=labels)
    loss = token_cross_entropy(logits, labels)
    backpropagate(loss)
    optimizer.step() when gradient accumulation reaches 8 micro-batches
    evaluate validation loss at each epoch end
    keep checkpoint with lowest eval_loss
```

5.3 ค่าตั้งที่บันทึก

พารามิเตอร์	ค่า
Base model	google/mt5-small
Epochs	1
Train / validation rows	3,893 / 486
Per-device batch	1
Gradient accumulation	8
Effective batch ที่ตั้งใจ	ประมาณ 8 samples ต่อ optimizer update
Learning rate	2×10^{-5}
Weight decay	0.01
Seed	42
Checkpoint policy	save/evaluate ทุก epoch, เลือก eval_loss ต่ำสุด, เก็บ checkpoint ล่าสุด/ดีที่สุดหนึ่งชุด

รายงานระบุ training runtime 9,914 วินาที (ประมาณ 2 ชั่วโมง 45 นาที), train loss 7.6032, validation eval loss 2.1857 และ validation runtime 265 วินาที. Training log บันทึก loss เป็นช่วงทุก 25 optimizer steps

Loss ต่ำไม่เท่ากับ **paraphrase** ถูกต้อง: โมเดลกำลังเรียนเลียนแบบ target สรุปที่อยู่ในข้อมูล ไม่ได้ถูกสอนให้รักษาทุกข้อเท็จจริงหรือเปลี่ยนระดับภาษาอย่างควบคุม

6. การประเมินผลและหลักฐาน

6.1 วิธีของ evaluate.py

1. อ่าน test split ทั้งหมด (ค่า default --max-samples 0 หมายถึงทั้งหมด)
2. สร้าง prompt จาก instruction + source; tokenize padding/truncation ที่ 512 tokens
3. แบ่ง batch ค่าเริ่มต้น 4; generate ได้ไม่เกิน 256 tokens, beam search 4, deterministic (do_sample=False)
4. คำนวณ corpus chrF เทียบ reference และค่าเฉลี่ย ROUGE-L F1
5. เขียนทุก prediction ไป artifacts/predictions.jsonl และ report ไป artifacts/evaluation_report.json

6.2 ผลใน report ที่ repository มี

ตัวชี้วัด	ผล	ความหมาย/ข้อจำกัด
test examples	486	จำนวน test rows ใน report
chrF	23.4734	character n-gram overlap; ไม่ใช่ semantic truthfulness
ROUGE-L F1	0.0622	longest-common-subsequence overlap; ต่ำเมื่อเทียบ reference แต่ตีความแยกจาก task mismatch

metrics นี้เทียบ mT5 กับ target ที่เป็น synthetic summarization เท่านั้น ไม่ได้ประเมิน Typhoon CLI, ความถูกต้องของข้อเท็จจริง, ความเทียบเท่าทางความหมาย หรือระดับภาษา ตัวเลข CLI สองประโยคเป็น smoke test ไม่ใช่การทดลองเชิงสถิติ

6.3 Human evaluation ที่ยังต้องทำ

แนวทางก่อนนำเสนอ: สุ่ม test examples, แสดง source/reference/model output แบบไม่ชี้แนะ, ให้ผู้ประเมินอย่างน้อยสองคนให้คะแนน fluency, meaning preservation และ style appropriateness ระดับ 1–5 พร้อมทำเครื่องหมาย factual drift (ใช่/ไม่ใช่), แล้วรายงานคะแนนแยก/ความเห็นไม่ตรงกัน ปัจจุบันยังไม่มีไฟล์ผล human evaluation ใน repository

7. CLI, Typhoon2 และ output guard

7.1 การสร้าง prompt และผล

app.py ใช้ AutoTokenizer/AutoModelForCausalLM ของ Typhoon2, ประกอบ system/user messages, แล้ว serialize ด้วย chat template ของ tokenizer เพื่อได้ input token IDs. Model generate ใช้ `do_sample=False`, `repetition_penalty=1.08`, `no_repeat_ngram_size=4` และ `max_new_tokens` ที่ผู้ใช้กำหนด (default 128). เนื่องจาก `do_sample=False` ผลจะ deterministic สำหรับ weights/prompt/config เดิม

หลัง generate จะตัด token IDs ของ prompt ออกจากลำดับรวมก่อน decode, ลบ special tokens, trim whitespace และเก็บเฉพาะย่อหน้าแรกเพื่อตัดหมายเหตุเกมที่ไม่ได้อาจสร้าง

7.2 Guard เชิง heuristic

ตรวจอะไร	วิธี	เมื่อพบปัญหา
คำตอบว่าง	strip แล้วตรวจ empty	คืนเหตุผล reject
prompt/sentinel รั่ว	หา <extra_id_, instruction:, context:, โปรดเรียบเรียง	ไม่แสดง output; ใช้ source
ความยาวผิดปกติ	reject เมื่อยาวกว่า $\max(2 \times \text{source}, \text{source} + 120)$	fallback เป็น source
ตัวเลข	ดึงเลขอารบิก/ไทย, normalize เลขไทย, comma, ทศนิยม .00; หาก source มี "เที่ยง" (ไม่ใช่เที่ยงคืน) เพิ่มค่า 12 ที่คาดหมาย	ต้องตรงลำดับตัวเลข ไม่เช่นนั้น fallback
คำสำคัญที่กำหนด	เปรียบเทียบ subset ของ วันนี้/พรุ่งนี้/เมื่อวาน/มะรืน/เที่ยงคืน/บาท/เปอร์เซ็นต์	หากเพิ่ม/หายไป fallback

Guard ไม่ได้ตรวจความหมายทั่วไปหรือข้อเท็จจริงทุกชนิด เช่น ชื่อคน หน่วยอื่น ข้อปฏิเสธ ความสัมพันธ์เชิงเหตุผล และเลขที่เขียนเป็นคำ จึงเป็นการลดความเสี่ยงบางกลุ่ม ไม่ใช่ formal guarantee

7.3 โหมด

CLI mode	คำสั่งที่ส่งให้โมเดล	หมายเหตุ
rewrite	เรียบเรียงใหม่	default
formal	เรียบเรียงเป็นภาษาทางการ	ไม่มี supervised formality labels ใน training data
semiformal	เรียบเรียงเป็นภาษากึ่งทางการ	ผลขึ้นกับ pretrained model/prompt
concise	เรียบเรียงให้กระชับ	ความกระชับอาจตัดรายละเอียด จึงควรตรวจ

8. วิธีติดตั้งและรัน

8.1 เตรียม environment

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements-train.txt
```

README แนะนำ Python 3.11/3.12; environment ที่ตรวจในเครื่องนี้เป็น Windows + Python 3.14.3, torch 2.14.1, transformers 4.57.6, datasets 4.8.5. ผลติดตั้งอาจต่างกันตาม Python/driver/GPU

8.2 เตรียมข้อมูลและฝึก baseline

```
python prepare_data.py
python train.py
python evaluate.py
```

prepare_data.py จะดาวน์โหลด source dataset ถ้ายังไม่มี cache. train.py ใช้ CPU ได้แต่ช้า; evaluate.py ใช้ test ครบ 486 ตัวอย่างเป็นค่าเริ่มต้น. อย่างอื่น train เข้าถ้าต้องการเพียงทดลอง CLI

8.3 ใช้งาน CLI

```
python app.py
python app.py --mode formal --text "พรุ่งนี้ระบบจะปิดปรับปรุงตั้งแต่ 10 โมงเช้าถึงเที่ยง"
python app.py --mode concise --text "บริษัทจะจัดส่งสินค้าภายใน 3 วันทำการหลังยืนยันการชำระเงิน"
```

ครั้งแรกจะโหลด Typhoon2 ประมาณ 6.5 GB ไปยัง Hugging Face cache; ข้อความ generation ใช้ CPU ได้แต่ช้ากว่า GPU. ถ้า guard ปฏิเสธผล โปรแกรมจะแสดงเหตุผลและข้อความต้นฉบับแทน

8.4 ไฟล์รายงาน

เตรียมข้อมูล → data/manifest.json; ฝึก mT5 → artifacts/training_report.json; ประเมิน mT5 → artifacts/evaluation_report.json และ artifacts/predictions.jsonl. การเรียก CLI ไม่เขียน report โดยอัตโนมัติ และไม่มี telemetry ของผลผู้ใช้

9. ข้อจำกัด ความเสี่ยง และงานต่อ

1. **Task mismatch:** source data เป็น summarization; target อาจตัด/เพิ่มรายละเอียด จึงไม่ใช่ gold paraphrase
2. **mT5 baseline** มีพฤติกรรมผิด: เคยสร้าง output ซ้ำและเปลี่ยนเวลา; ไม่ใช่เป็นโมเดล CLI ปัจจุบัน
3. **Typhoon** ไม่ได้ **fine-tune** ด้วย **project data:** ทำให้ CLI ทดลองได้ดีขึ้น แต่ผลนี้ไม่ใช่ผลการฝึกจาก WangchanX dataset
4. **ไม่มี controlled-style labels:** formal/semiformal/concise เป็น prompt modes ไม่ใช่ความสามารถที่วัดรับรอง
5. **Guard** มีขอบเขตจำกัด: heuristics ตรวจเฉพาะ marker, length, ตัวเลข และคำสำคัญบางคำ; ไม่รับรอง semantic equivalence
6. **ผลประเมินคนละโมเดล:** metrics ใน report มาจาก mT5 บน summarization targets; ห้ามนำไปอ้างว่า Typhoon CLI ได้คะแนนดังกล่าว
7. **ทรัพยากร:** Typhoon2 ใช้ RAM/เวลาเพิ่ม; เครื่องที่ไม่มี CUDA จะทำงานบน CPU

งานต่อที่มีหลักฐานรองรับ

- หา/จัดทำ Thai paraphrase pairs ที่มี source, target, style labels และ license ตรวจสอบได้; อย่าแปลง QA/brainstorming เป็นคู่ paraphrase โดยไม่มี annotation
- ทำ train/validation/test สำหรับ task ใหม่โดย split ตาม source/document group ป้องกัน leakage
- สร้าง regression set ที่มีตัวเลข เวลา วันที่ หน่วย ชื่อ และ negation; ตรวจ exact preservation และ human review
- ประเมิน Typhoon CLI แยกจาก mT5 ด้วย sample set เดียวกันและบันทึก model ID/revision, prompt, decoding params
- รายงาน human ratings และ error analysis ก่อนกล่าวอ้างว่าใช้งานเชื่อถือได้

10. ที่มาและการอ้างอิง

1. Pengpun, Parinthapat, Can Udomcharoenchaikit, Weerayut Buaphet, and Peerat Limkonchotiawat. 2024. "Seed-Free Synthetic Data Generation Framework for Instruction-Tuning LLMs: A Case Study in Thai." ACL Student Research Workshop. aclanthology.org/2024.acl-srw.38.
2. Dataset: [airesearch/wangchanx-seed-free-synthetic-instruct-thai-120k](https://github.com/airesearch/wangchanx-seed-free-synthetic-instruct-thai-120k). manifest ระบุ MIT ตาม dataset card; ตรวจสอบ card ฉบับปัจจุบันก่อนนำไปเผยแพร่
3. CLI model: [typhoon-ai/llama3.2-typhoon2-3b-instruct](https://github.com/typhoon-ai/llama3.2-typhoon2-3b-instruct). model card ระบุ Thai/English และ Llama 3.2 Community License; ต้องตรวจสอบเงื่อนไขการใช้ก่อนแจกจ่าย/เผยแพร่
4. Baseline model: [google/mt5-small](https://ai.google.dev/models/mt5-small).

แนวทางนำเสนอ: ให้เรียก mT5 ว่า "baseline fine-tuned บน weakly supervised summarization pairs"; เรียก Typhoon2 ว่า "pretrained instruction model ที่ใช้เป็น CLI prototype"; ระบุอย่างตรงไปตรงมาว่ายังไม่มี Thai gold paraphrase dataset และยังไม่มี human-evaluation result

ไฟล์นี้อธิบาย repository ตามสภาพ ณ วันที่จัดทำ ไม่ได้แทน license text, model card, dataset card หรือการตรวจสอบทางวิชาการฉบับเต็ม